

PNG LSB-Стеганография — Постановка задачи и описание алгоритма

Роль: Постановщик задачи / Разработчик алгоритма

Проект: [steganography](#)

Версия документа: 1.0

Статус проекта: 1.0-SNAPSHOT (в разработке)

Язык реализации: Python 3.13, PyQt5

Содержание

1. [Введение и общая концепция](#)
 2. [Предметная область — что такое стеганография](#)
 3. [Постановка задачи](#)
 4. [Выбор метода и его обоснование](#)
 5. [Технические ограничения и требования к входным данным](#)
 6. [Детальное описание алгоритма LSB в палитре PNG](#)
 7. [Формат встраиваемых данных](#)
 8. [Алгоритм встраивания \(Embed / Encrypt\)](#)
 9. [Алгоритм извлечения \(Extract / Decrypt\)](#)
 10. [Расчёт ёмкости канала](#)
 11. [Оценка визуальной незаметности](#)
 12. [Архитектура приложения](#)
 13. [Описание тестового покрытия](#)
 14. [Ограничения метода и возможные улучшения](#)
 15. [Глоссарий](#)
-

1. Введение и общая концепция

Данный документ описывает концептуальную и алгоритмическую основу учебного проекта по **стеганографии** — науке о скрытой передаче информации. Документ составлен с позиции постановщика задачи и разработчика алгоритма и охватывает:

- обоснование выбора предметной области и метода,
- формальное описание алгоритма встраивания и извлечения данных,
- технические ограничения и расчёты ёмкости,
- архитектурные решения, принятые в ходе проектирования,
- полное описание тестового покрытия.

Общая идея приложения

Приложение представляет собой **кроссплатформенное десктопное GUI-приложение** на Python + PyQt5, которое позволяет пользователю:

1. Загрузить PNG-изображение с диска.
2. Ввести текстовое сообщение (секрет).
3. Незаметно спрятать сообщение внутри изображения.
4. Сохранить «стего-изображение» — файл, внешне неотличимый от оригинала.
5. В любой момент извлечь спрятанное сообщение из файла.

Соккрытие происходит **исключительно в палитре PNG** (чанк `PLTE`), без изменения пиксельных данных изображения — это принципиальное архитектурное решение.

2. Предметная область — что такое стеганография

Стеганография (от греч. *στεγανός* — скрытый, *γράφω* — пишу) — это искусство и наука о скрытой передаче информации путём сокрытия самого факта передачи. В отличие от **криптографии**, которая скрывает *содержимое* сообщения (шифрование), стеганография скрывает *существование* сообщения.

Сравнение подходов

Свойство	Криптография	Стеганография
Что скрывается	Содержание сообщения	Сам факт существования сообщения
Обнаружимость	Факт передачи очевиден	Факт передачи скрыт
Защита при обнаружении	Высокая (ключ неизвестен)	Низкая (после обнаружения данные читаемы)
Типичный носитель	Любой канал	Мультимедиа (изображения, аудио, видео)
Сочетаемость	Можно комбинировать	Можно комбинировать с шифрованием

Область применения стеганографии

- **Цифровые водяные знаки** — защита авторских прав (DRM).
- **Скрытые каналы связи** — передача данных в условиях цензуры.
- **Идентификация источника утечки** — отпечатки (fingerprinting).
- **Образовательные цели** — изучение методов обработки изображений.

В данном учебном проекте стеганография реализована **без шифрования**: сообщение встраивается в открытом виде, что соответствует образовательным целям и упрощает понимание механизма.

3. Постановка задачи

3.1 Формулировка задачи

Разработать программный инструмент, реализующий **метод LSB (Least Significant Bit)** для скрытия текстовых сообщений в **палитре PNG-изображений** (чанк PLTE, тип цвета ColorType = 3).

3.2 Функциональные требования

№	Требование	Приоритет
FR-01	Загрузка PNG-изображения из файловой системы	Обязательно
FR-02	Автоматическое приведение любого PNG к формату Indexed8 (256-цветная палитра)	Обязательно
FR-03	Встраивание текстового сообщения (UTF-8) в LSB-биты RGB-компонентов палитры	Обязательно
FR-04	Извлечение скрытого сообщения из стего-изображения	Обязательно
FR-05	Сохранение стего-изображения в PNG-файл с сохранением палитры	Обязательно
FR-06	Визуальное сравнение исходного и стего-изображений	Обязательно
FR-07	Отображение извлечённого сообщения в интерфейсе	Обязательно
FR-08	Проверка ёмкости палитры до встраивания	Обязательно
FR-09	Поддержка Unicode (кириллица, эмодзи и др.) через UTF-8	Желательно
FR-10	Информирование пользователя через статусную строку	Желательно

3.3 Нефункциональные требования

№	Требование	Значение
NFR-01	Язык программирования	Python 3.13
NFR-02	GUI-фреймворк	PyQt5
NFR-03	Кроссплатформенность	Windows, Linux
NFR-04	Дистрибуция	PyInstaller (.exe / бинарный файл)
NFR-05	Максимальное визуальное отклонение	1/255 $\approx$ 0.4% яркости на канал
NFR-06	Тестирование	pytest + pytest-qt, $\geq$ 12 групп тестов
NFR-07	Объём кода	main.py ~1200 строк, tests.py ~1500 строк

3.4 Ограничения

- Метод работает **только с PNG-изображениями** (формат с поддержкой палитры PLTE).
- Для встраивания обязательно наличие **цветовой палитры** (Format_Indexed8).
- Максимальная длина сообщения ограничена **размером палитры**: 96 байт для стандартной 256-цветной палитры.
- Формат сохранения — исключительно **PNG** (JPEG и BMP уничтожают палитру).

4. Выбор метода и его обоснование

4.1 Почему LSB?

Метод **LSB (Least Significant Bit)** — замена младшего бита каждого байта носителя на бит скрываемого сообщения — является наиболее фундаментальным и широко изученным методом стеганографии. Он выбран по следующим причинам:

- Простота реализации.** Алгоритм сводится к битовым операциям AND и OR, доступным на любом уровне абстракции.
- Визуальная незаметность.** Изменение младшего бита меняет значение пикселя максимум на 1 из 255 (~0.4%), что находится за порогом зрительного восприятия человека.
- Детерминированность.** Одни и те же входные данные всегда дают одинаковый результат — это важно для тестирования и верификации.
- Образовательная ценность.** Метод наглядно демонстрирует принцип сокрытия информации и базовую работу с битами.

4.2 Почему палитра PLTE, а не пиксели?

Это ключевое архитектурное решение, отличающее данную реализацию от большинства учебных примеров.

Подход	Встраивание в пиксели	Встраивание в палитру PLTE
Объём носителя	Зависит от размера изображения (потенциально огромный)	Фиксирован: $256 \times 3 = 768$ байт
Видимость изменений	Меняются сами пиксели	Пиксельные данные не меняются
Стойкость к сжатию	Низкая (пересжатие разрушает данные)	Высокая (палитра хранится отдельным чанком)
Сложность реализации	Средняя	Низже (работаем только с таблицей цветов)
Максимальная ёмкость	Большая	Ограничена 94 байтами текста

Встраивание в палитру — **более чистое** с точки зрения архитектуры: пиксельные данные изображения остаются полностью нетронутыми. Изменяется только таблица цветов, которая хранится отдельно в чанке `PLTE`.

4.3 Почему PNG с ColorType = 3?

PNG-формат поддерживает несколько типов цвета. Для реализации необходим тип **ColorType = 3** (indexed color / палитровый):

```
ColorType = 0  - Grayscale (нет палитры)
ColorType = 2  - TrueColor RGB (нет палитры)
```

```
ColorType = 3  — Indexed color (ЕСТЬ палитра PLTE) ← используем
ColorType = 4  — Grayscale + Alpha (нет палитры)
ColorType = 6  — TrueColor + Alpha (нет палитры)
```

Только `ColorType = 3` содержит чанк `PLTE` — таблицу от 1 до 256 RGB-цветов, которая и является носителем скрытой информации.

5. Технические ограничения и требования к входным данным

5.1 Требования к изображению-носителю

Изображение должно быть в формате PNG. При загрузке нецветовой PNG (например, RGB24, RGBA32) приложение автоматически конвертирует его в `Format_Indexed8` через алгоритм квантования цветов Qt (**медианное разрезание**, до 256 цветов). Это **разрушающая операция**: рекомендуется использовать изначально палитровые PNG.

Рекомендуемые параметры изображения:

- Формат файла: `.png`
- Тип цвета: `ColorType = 3` (Indexed color, 256-цветная палитра)
- Размер: любой (на ёмкость влияет только размер палитры)

5.2 Требования к сообщению

- Кодировка: **UTF-8** (поддерживается любой язык, включая эмодзи)
- Максимальная длина: ограничена ёмкостью палитры (см. раздел 10)
- Тип данных: **текст** (бинарные данные не поддерживаются в текущей версии)
- Максимальная теоретическая длина заголовка: 65 535 байт (`uint16`), но на практике ограничена размером палитры

6. Детальное описание алгоритма LSB в палитре PNG

6.1 Структура PNG и чанк PLTE

PNG-файл состоит из последовательности **чанков** (chunks). Каждый чанк имеет структуру:

```
[4 байта: длина данных] [4 байта: тип чанка] [N байт: данные] [4 байта: CRC]
```

Для палитрового PNG ключевые чанки:

```
IHDR — заголовок (ширина, высота, глубина цвета, ColorType=3, ...)
PLTE — палитра: от 1 до 256 записей по 3 байта (R, G, B)
IDAT — сжатые пиксельные данные (каждый пиксель = индекс в палитре)
IEND — маркер конца файла
```

Чанк `PLTE` — это и есть носитель скрытой информации. Каждая запись палитры хранит тройку (R, G, B) , где каждый компонент занимает 1 байт (значение 0–255).

6.2 Принцип LSB

В каждом байте RGB-компонента есть 8 битов: от **бита 7 (MSB, наиболее значащий)** до **бита 0 (LSB, наименее значащий)**. Изменение LSB меняет число на ± 1 :

Пример:

```
Исходный байт:      10110110  = 182
Изменение LSB:      10110111  = 183  (+1)
Изменение LSB:      10110110  = 182  (без изменений)
```

Разница: максимум 1 из 255 $\approx 0.39\%$

Алгоритм заменяет LSB каждого байта R, G, B в каждой записи палитры на соответствующий бит скрываемого сообщения. Таким образом, каждая запись палитры вмещает **3 бита** скрытой информации.

6.3 Схема распределения битов по палитре

Битовый поток скрытых данных: `b0 b1 b2 b3 b4 b5 b6 b7 b8 ...`

Палитра:

```
Color[0]:  R0 → LSB = b0
           G0 → LSB = b1
           B0 → LSB = b2

Color[1]:  R1 → LSB = b3
           G1 → LSB = b4
           B1 → LSB = b5

Color[2]:  R2 → LSB = b6
           G2 → LSB = b7
           B2 → LSB = b8

...
```

Биты идут от старшего к младшему внутри каждого байта данных (big-endian порядок битов).

6.4 Работа с Qt: представление цвета QRgb

В PyQt5 цвет палитры хранится в формате `QRgb` — 32-битное целое число:

Формат: 0xAARRGGBB

Биты 31..24 → AA (альфа-канал, всегда 0xFF для непрозрачного)
Биты 23..16 → RR (красный канал, 0..255)
Биты 15..8 → GG (зелёный канал, 0..255)
Биты 7..0 → BB (синий канал, 0..255)

Для работы с каналами реализованы вспомогательные функции:

```
def qRed(rgb: int) -> int:
    return (rgb >> 16) & 0xFF          # Извлечь красный канал

def qGreen(rgb: int) -> int:
    return (rgb >> 8) & 0xFF          # Извлечь зелёный канал

def qBlue(rgb: int) -> int:
    return rgb & 0xFF                  # Извлечь синий канал

def qRgb(r: int, g: int, b: int) -> int:
    return (0xFF << 24) | (r << 16) | (g << 8) | b  # Собрать цвет
```

7. Формат встраиваемых данных

Перед встраиванием текстовое сообщение преобразуется в **бинарный поток** следующей структуры:

2 байта	:	длина сообщения (uint16, big-endian)
N байт	:	тело сообщения в кодировке UTF-8

7.1 Заголовок длины

Первые 2 байта кодируют длину сообщения как **беззнаковое 16-битное целое в формате big-endian** (сначала старший байт):

```
length_bytes = struct.pack('>H', len(msg_bytes))
# '>H' — big-endian unsigned short (2 байта)
# Диапазон: 0..65535
```

Заголовок длины необходим для точного определения границы сообщения при извлечении — без него алгоритм не знает, где заканчивается полезная нагрузка.

7.2 Тело сообщения

Текст кодируется в **UTF-8**, что позволяет работать с любым Unicode-текстом: латиницей, кириллицей, китайскими иероглифами, эмодзи и другими символами.

```
msg_bytes = message.encode('utf-8')
```

Важно: UTF-8 кодирует символы переменным числом байт:

- ASCII-символы (0x00..0x7F): 1 байт
- Кириллица, греческий, и др.: 2 байта
- Китайский, японский, и др.: 3 байта
- Эмодзи и редкие символы: 4 байта

Это означает, что максимальное число символов зависит от их типа.

7.3 Итоговый поток данных

```
data = bytearray()
data.extend(struct.pack('>H', length))    # 2 байта заголовка
data.extend(msg_bytes)                   # N байт текста

# Итого: (2 + N) * 8 бит должны поместиться в len(palette) * 3 бит
```

8. Алгоритм встраивания (Embed / Encrypt)

8.1 Входные данные

- `original_image` — исходное PNG-изображение формата `QImage.Format_Indexed8`
- `message` — текстовая строка для встраивания (непустая)

8.2 Выходные данные

- `stego_image` — копия исходного изображения с модифицированной палитрой

8.3 Описание алгоритма

АЛГОРИТМ EMBED (встраивание):

Вход:

```
image    — QImage (Format_Indexed8)
message  — строка (UTF-8)
```

1. Закодировать сообщение:

```
msg_bytes ← message.encode('utf-8')
length    ← len(msg_bytes)
```

```

2. Сформировать бинарный поток данных:
    data ← struct.pack('>H', length) + msg_bytes
    (2 байта заголовка + N байт текста)

3. Проверить ёмкость палитры:
    colors ← image.colorTable()          // Список QRgb
    max_bits ← len(colors) × 3
    need_bits ← (2 + length) × 8
    ЕСЛИ need_bits > max_bits:
        ОШИБКА "Сообщение не помещается в палитру"

4. Создать рабочую копию:
    img ← image.copy()
    colors ← img.colorTable()

5. LSB-встраивание:
    bit_index ← 0
    total_bits ← len(data) × 8

    ДЛЯ i = 0 ДО len(colors)-1:
        r ← qRed(colors[i])
        g ← qGreen(colors[i])
        b ← qBlue(colors[i])

        ЕСЛИ bit_index < total_bits:
            byte_idx ← bit_index // 8
            bit_in_byte ← 7 - (bit_index % 8)
            bit ← (data[byte_idx] >> bit_in_byte) & 1
            r ← (r & 0xFE) | bit      // Заменить LSB красного
            bit_index ← bit_index + 1

        ЕСЛИ bit_index < total_bits:
            (аналогично для g)
            bit_index ← bit_index + 1

        ЕСЛИ bit_index < total_bits:
            (аналогично для b)
            bit_index ← bit_index + 1

        colors[i] ← qRgb(r, g, b)

    ЕСЛИ bit_index ≥ total_bits: ПРЕРВАТЬ ЦИКЛ

6. Применить изменённую палитру:
    img.setColorTable(colors)

7. Сохранить результат:
    stego_image ← img

Выход: stego_image

```

8.4 Ключевые операции над битами

Замена LSB выполняется двумя битовыми операциями:

```
# Шаг 1: обнулить LSB (маска 0xFE = 11111110)
r_cleared = r & 0xFE

# Шаг 2: установить нужный бит
r_new = r_cleared | bit # bit ∈ {0, 1}
```

Это эквивалентно: $r_new = (r \& 0xFE) | bit$

Извлечение бита из потока данных:

```
byte_idx    = bit_index // 8          # В каком байте находится нужный бит
bit_in_byte = 7 - (bit_index % 8)    # Смещение от LSB (7 = MSB, 0 = LSB)
bit         = (data[byte_idx] >> bit_in_byte) & 1 # Извлечённый бит
```

8.5 Инвариант алгоритма

После встраивания гарантируется:

1. Исходное изображение (`original_image`) **не изменяется** — работа ведётся с глубокой копией.
2. Пиксельные данные (индексы цветов) **не изменяются** — изменяется только таблица цветов.
3. Размер палитры **не изменяется** — количество записей остаётся прежним.
4. Изменены только **биты 0 (LSB)** каждого RGB-компонента — биты 7..1 не тронуты.

9. Алгоритм извлечения (Extract / Decrypt)

9.1 Входные данные

- `stego_image` (приоритет) или `original_image` — PNG-изображение с возможно встроенным сообщением

9.2 Выходные данные

- `extracted_message` — строка, извлечённая из LSB-битов палитры

9.3 Описание алгоритма

АЛГОРИТМ EXTRACT (извлечение):

Вход:

`image` — QImage (Format_Indexed8), содержащее стего-данные

```

1. Получить палитру:
    colors ← image.colorTable()

2. Собрать битовый поток из LSB:
    bits ← []
    ДЛЯ КАЖДОГО color В colors:
        bits.append(qRed(color) & 1)    // LSB красного
        bits.append(qGreen(color) & 1)   // LSB зелёного
        bits.append(qBlue(color) & 1)    // LSB синего

    // Итого: len(colors) × 3 бит

3. Преобразовать биты в байты:
    data_bytes ← []
    ДЛЯ i = 0 ШАГОМ 8 ДО len(bits)-7:
        byte_val ← 0
        ДЛЯ j = 0 ДО 7:
            byte_val ← (byte_val << 1) | bits[i + j]
        data_bytes.append(byte_val)

4. Считать заголовок длины (первые 2 байта):
    length ← struct.unpack('>H', data_bytes[0:2])[0]

    ЕСЛИ length > len(data_bytes) - 2:
        ОШИБКА "Некорректные данные / изображение не содержит сообщения"

5. Извлечь тело сообщения:
    msg_bytes ← data_bytes[2 : 2 + length]
    message ← msg_bytes.decode('utf-8')

6. Отобразить результат:
    extracted_output ← message

Выход: message

```

9.4 Приоритет источника изображения

При извлечении используется следующая логика выбора источника:

```
target = self.stego_image if self.stego_image is not None else self.original_image
```

Это позволяет извлекать сообщения как из только что созданного стего-изображения, так и из загруженного с диска PNG-файла.

10. Расчёт ёмкости канала

10.1 Формула ёмкости

Максимальная ёмкость (в битах):

```
max_bits = N_colors × 3
```

Максимальная ёмкость (в байтах):

```
max_bytes = (N_colors × 3) / 8 = N_colors × 0.375
```

Максимальная длина текста (в байтах):

```
max_text = max_bytes - 2          (минус 2 байта заголовка)  
          = N_colors × 0.375 - 2
```

10.2 Таблица ёмкости для разных размеров палитры

Размер палитры	Бит всего	Байт всего	Максимум текста (ASCII)	Максимум текста (кириллица)
8 цветов	24 бит	3 байта	1 символ	невозможно
16 цветов	48 бит	6 байт	4 символа	2 символа
32 цвета	96 бит	12 байт	10 символов	5 символов
64 цвета	192 бит	24 байт	22 символа	11 символов
128 цветов	384 бит	48 байт	46 символов	23 символа
256 цветов	768 бит	96 байт	~94 символа	~47 символов

10.3 Стандартная конфигурация

Для стандартной 256-цветной палитры:

```
max_bits   = 256 × 3 = 768 бит  
max_bytes  = 768 / 8 = 96 байт  
max_text   = 96 - 2  = 94 байта
```

При ASCII-тексте (1 байт/символ): до 94 символов

При кириллице (2 байта/символ): до 47 символов

При китайском (3 байта/символ): до 31 символа

10.4 Проверка ёмкости в коде

```
max_bits   = len(colors) * 3  
needed_bits = (2 + length) * 8  
  
if needed_bits > max_bits:  
    available_text_bytes = max(0, max_bits // 8 - 2)  
    # Показываем ошибку с диагностической информацией
```

11. Оценка визуальной незаметности

11.1 Теоретический анализ

Изменение LSB меняет значение байта на ± 1 . Для RGB-компонентов в диапазоне [0, 255]:

Максимальное абсолютное изменение:	1 (из 255 возможных)
Относительное изменение:	$1/255 \approx 0.392\%$

Человеческий глаз обнаруживает изменения яркости только при разнице не менее **~5-10 уровней** (зависит от условий наблюдения). Изменение на 1 уровень из 255 находится **значительно ниже** порога восприятия.

11.2 Особенность встраивания в палитру

В данной реализации изменяются **записи палитры**, а не сами пиксели. Один и тот же цвет палитры может использоваться тысячами пикселей. Это значит:

- Если цвет (100, 100, 100) изменяется на (101, 100, 100), то **все пиксели**, ссылающиеся на этот цвет, изменятся одновременно.
- Тем не менее, изменение на 1 уровень по одному каналу по-прежнему практически незаметно.

11.3 Рекомендации для лучшей незаметности

Для максимальной визуальной незаметности рекомендуется использовать:

- Изображения с большим количеством **средних тонов** (серые, бежевые), где изменение ± 1 наименее заметно.
- Изображения с **шумом или текстурой** (фотографии природы, тканей), а не однотонные.
- Изображения, не прошедшие повторного сжатия после встраивания.

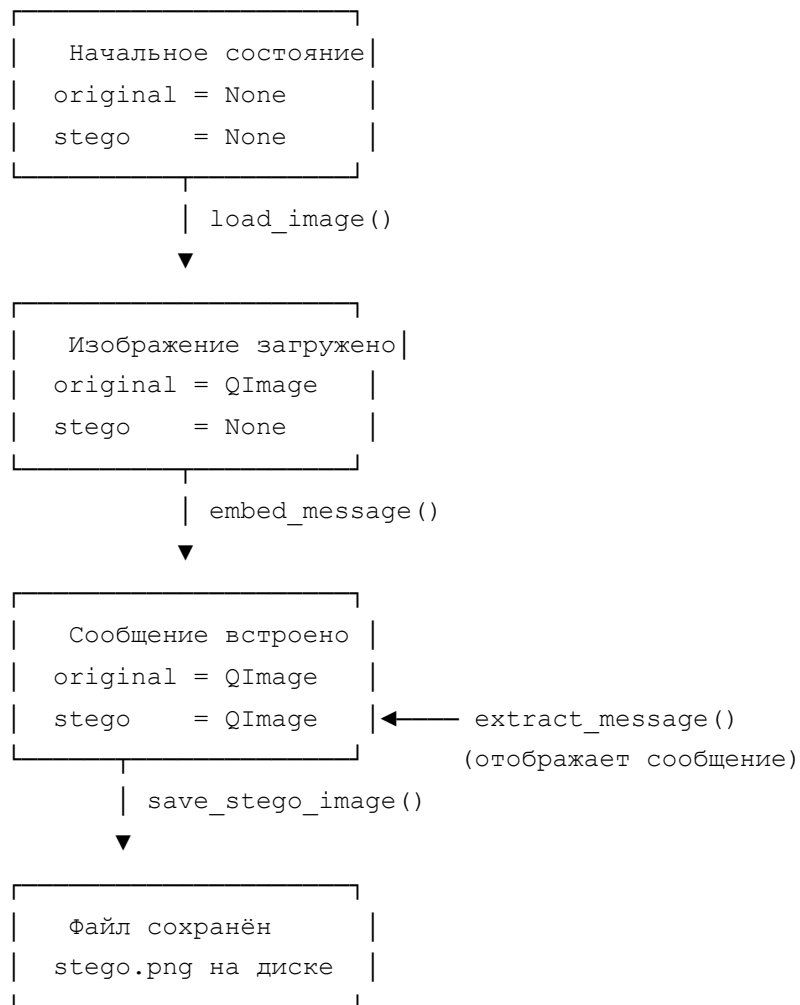
12. Архитектура приложения

12.1 Структура файлов

```
steganography/
├─ main.py           — Основная программа (~1227 строк)
├   └─ Вспомогательные функции: qRed, qGreen, qBlue, qRgb
├   └─ Константа: APP_STYLE (глобальная таблица QSS-стилей)
├   └─ Класс: SteganoWindow(QMainWindow)
├       └─ __init__()           — инициализация окна
├       └─ _build_ui()          — построение интерфейса
├       └─ _make_header()       — шапка приложения
├       └─ _make_original_group() — панель исходного изображения
├       └─ _make_stego_group()  — панель стего-изображения
├       └─ _make_input_group()  — поле ввода сообщения
├       └─ _make_output_group() — поле вывода сообщения
├       └─ _make_action_row()   — кнопки действий
├       └─ _refresh_buttons()   — управление состоянием кнопок
```

- | | | | load_image() - загрузка PNG из файла
- | | | | save_stego_image() - сохранение PNG в файл
- | | | | embed_message() - АЛГОРИТМ ВСТРАИВАНИЯ
- | | | | extract_message() - АЛГОРИТМ ИЗВЛЕЧЕНИЯ
- | | | | _show_image() - отображение QImage в QLabel
- |
- | tests.py - Тесты (~1492 строки, 12 групп)
- | requirements.txt - Зависимости
- | images/ - Тестовые изображения
- | win/ - Собранный .exe для Windows
- | | README.md - Документация проекта

12.2 Схема состояний приложения



12.3 Управление состоянием кнопок (_refresh_buttons)

Кнопки блокируются/разблокируются автоматически в зависимости от наличия изображений:

Кнопка	Условие активации
Загрузить	Всегда активна
Встроить	<code>original_image is not None</code>
Извлечь	<code>original_image is not None</code> OR <code>stego_image is not None</code>
Сохранить	<code>stego_image is not None</code>

12.4 Дизайн интерфейса

Приложение выполнено в стиле **Catppuccin Mocha** (тёмная тема) с характерной цветовой схемой:

Фон приложения:	#1e1e2e	(тёмно-синий)
Фон групп:	#313244	(тёмно-серый)
Кнопка «Загрузить»:	#89b4fa	(голубой)
Кнопка «Встроить»:	#a6e3a1	(зелёный)
Кнопка «Извлечь»:	#fab387	(оранжевый)
Кнопка «Сохранить»:	#c8a6f7	(фиолетовый)
Текст:	#cdd6f4	(светло-серый)

Интерфейс разделён на **4 зоны**:

1. Шапка с названием и описанием метода
2. Два блока предпросмотра изображений (исходное и стего)
3. Два текстовых поля (ввод и вывод сообщения)
4. Кнопки действий и статусная строка

13. Описание тестового покрытия

Файл `tests.py` содержит **12 групп тестов** с маркерами `unit`, `ui`, `integration`, `edge_cases`, `performance`.

13.1 Обзор групп тестов

Группа	Класс	Тип	Описание	Примерное число тестов
1	TestColorHelperFunctions	unit	Функции qRed/qGreen/qBlue/qRgb	~18
2	TestLSBAlgorithmCore	unit	Чистая логика LSB без Qt	~10
3	TestUIInitialization	ui	Начальное состояние окна	~18
4	TestButtonStates	ui	Логика блокировки кнопок	~11
5	TestEmbedMessage	ui	Метод embed_message()	~13
6	TestExtractMessage	ui	Метод extract_message()	~6
7	TestFullEmbedExtractCycle	integration	Полный цикл embed → extract	~9
8	TestFileOperations	integration	Сохранение и загрузка файлов	~5
9	TestEdgeCases	edge_cases	Граничные значения	~8
10	TestErrorHandling	ui	Обработка ошибочных состояний	~6
11	TestUnicodeMessages	unit	Unicode, UTF-8, многобайтовые	~8
12	TestPerformance	performance	Скорость операций	~4

13.2 Ключевые тестовые сценарии

Группа 1 — вспомогательные функции:

- `qRgb(255, 255, 255) → 0xFFFFFFFF`
- `qRed(qRgb(123, 45, 67)) → 123`
- Симметрия: `qRgb(qRed(c), qGreen(c), qBlue(c)) == c`

Группа 2 — ядро алгоритма LSB:

- После встраивания изменены **только LSB** (биты 7..1 нетронуты)
- Встраивание ASCII → извлечение даёт оригинал
- Встраивание кириллицы → извлечение даёт оригинал
- Заголовок длины содержит корректное значение
- Сообщение точно в размер палитры — успешно (граничный случай)
- Сообщение на 1 байт больше ёмкости — `ValueError`

Группа 7 — полный цикл:

- ASCII короткий / длинный (до ёмкости)
- Кириллица, цифры, знаки препинания, переносы строк
- Сохранение в файл → загрузка → извлечение (проверка персистентности)

Группа 11 — Unicode:

- Кириллица, смешанный текст
- Пустая строка граничный случай
- Символы с кодовыми точками > U+007F

13.3 Фикстуры pytest

Фикстура	Область (scope)	Описание
<code>qapp</code>	<code>session</code>	Единственный экземпляр <code>QApplication</code> для всех тестов
<code>window</code>	<code>function</code>	Свежий экземпляр <code>SteganoWindow</code> для каждого теста
<code>palette_256</code>	<code>function</code>	Список из 256 серых <code>QRgb</code> -значений
<code>palette_small</code>	<code>function</code>	Список из 8 <code>QRgb</code> -значений (минимальная ёмкость)
<code>indexed8_image_256</code>	<code>function</code>	<code>QImage</code> 10×10, <code>Format_Indexed8</code> , 256 цветов
<code>indexed8_image_small</code>	<code>function</code>	<code>QImage</code> 4×4, <code>Format_Indexed8</code> , 8 цветов
<code>tmp_png</code>	<code>function</code>	Временный PNG-файл на диске (автоматически удаляется)

13.4 Запуск тестов

```
# Установить зависимости
pip install -r requirements.txt

# Все тесты
pytest tests.py -v
```

```
# Только юнит-тесты
pytest tests.py -v -m unit

# Только UI-тесты
pytest tests.py -v -m ui

# Только интеграционные
pytest tests.py -v -m integration

# С кратким выводом ошибок
pytest tests.py -v --tb=short

# С отчётом о покрытии (если установлен pytest-cov)
pytest tests.py -v --cov=main --cov-report=term-missing
```

14. Ограничения метода и возможные улучшения

14.1 Текущие ограничения

- Ёмкость.** Максимум ~94 байт текста для стандартной 256-цветной палитры — это небольшой объём. Для длинных сообщений метод непригоден без расширения.
- Только PNG.** JPEG не поддерживает палитровый режим и использует сжатие с потерями. BMP поддерживает палитру, но не имеет структуры чанков.
- Отсутствие шифрования.** Любой, кто знает о применённом методе, может извлечь сообщение. Необходимо дополнительное шифрование для практического применения.
- Стойкость к анализу.** Метод уязвим к стегоанализу: статистический анализ LSB-бит палитры выявит неестественное распределение.
- Формат Indexed8.** Изображения с небольшой палитрой (менее 256 цветов) имеют ещё меньшую ёмкость.

14.2 Возможные улучшения

Улучшение	Описание	Сложность
Шифрование AES	Шифровать сообщение перед встраиванием	Средняя
Встраивание в пиксели	Дополнительный канал — пиксельные данные	Средняя
2-bit LSB	Использовать 2 младших бита (менее незаметно, больше ёмкость)	Низкая
Поддержка бинарных данных	Встраивать файлы, а не только текст	Средняя
Стегоанализ	Проверка наличия скрытых данных	Высокая
Пароль на извлечение	Требовать ключ для расшифровки	Средняя
Прогрессбар	Отображение прогресса для больших изображений	Низкая
Drag & Drop	Перетаскивание PNG-файла в окно	Низкая
История операций	Undo/redo для встраивания	Высокая

15. Глоссарий

Термин	Определение
Стеганография	Наука о скрытой передаче информации путём сокрытия самого факта её существования
LSB	Least Significant Bit — наименее значащий (младший) бит числа
PNG	Portable Network Graphics — формат изображений с поддержкой палитры и без потерь
PLTE	Palette chunk — чанк палитры в PNG-файле, содержащий до 256 записей RGB
ColorType = 3	Тип цвета PNG, обозначающий палитровое (indexed) изображение
Format_Indexed8	Константа Qt, обозначающая 8-битное палитровое изображение
QRgb	32-битное целое Qt для хранения цвета в формате 0xAARRGGBB
Стего-изображение	Изображение со встроенным скрытым сообщением
Носитель	Исходное изображение, в которое встраивается информация
Ёмкость канала	Максимальный объём информации, который можно скрыть в носителе
UTF-8	Кодировка Unicode с переменной длиной символа (1–4 байта)
Big-endian	Порядок байт, при котором старший байт записывается первым
uint16	Беззнаковое 16-битное целое (диапазон 0..65535)
Квантование цветов	Уменьшение числа цветов изображения до ≤ 256 для создания палитры
Медианное разрезание	Алгоритм квантования цветов, используемый Qt при конвертации в Indexed8
PyQt5	Python-привязка к библиотеке Qt 5 для создания GUI-приложений
PyInstaller	Инструмент для упаковки Python-приложений в исполняемые файлы
pytest	Фреймворк для тестирования Python-кода
pytest-qt	Плагин pytest для тестирования PyQt5-приложений
QSS	Qt Style Sheets — язык стилизации виджетов Qt (аналог CSS)

Команда разработчиков

Участник	GitHub	Роль
Гущина Маргарита	Oycea	Руководитель проекта, документовед
Разумов Михаил	Nel4nne	Дизайнер GUI-интерфейса
Слелин Александр	a-slelin	Программист, тестировщик
Гаджигасанов Тимур	soulnier	Постановщик задачи, разработчик алгоритма

Документ подготовлен в рамках учебного проекта. Версия 1.0. Статус: актуально для версии 1.0-SNAPSHOT.